Exercise 1

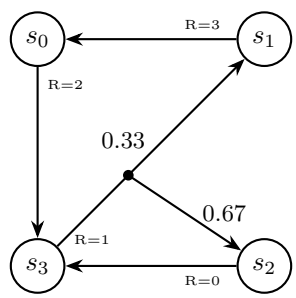
a) Given CID = 01897694. After deleting the leading zero, we get CID = 1897694 $\Rightarrow$ CID(1) = 1, CID(2) = 8, etc. For the first state, we have

$$s(1) = (CID(1) + 2)\%4 = 3\%4 = 3, r(1) = 1\%4 = 1 \Rightarrow (s3, +1)$$

If we apply this formula to the whole CID, we get the following trace:

$$(s3, +1) \Rightarrow (s2, 0) \Rightarrow (s3, +1) \Rightarrow (s1, +3) \Rightarrow (s0, +2) \Rightarrow (s3, +1) \Rightarrow (s2)$$

b) Using the data obtained in part a, I drew an edge from any predecessor state to its successor state. However, since it is given that there is only one possible action at each step and that transitions depend only on transition dynamics and not actions, I drew only one arrow coming out of each state. This arrow splits and goes into the different states that could be reached from the original state. I explain how I got the transition probabilities and rewards in part c.



c) 1) The rows and columns in the probability matrix represent states 0 to 3. The probabilities in any element (row, col) represent how many times we transitioned from state row to state column out of all the times we transitioned out of state row (probability).

$$P = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{1}{3} & \frac{2}{3} & 0 \end{bmatrix}$$

2) The reward is associated with leaving the state, and not with taking a particular action. For example, if we are in state s3, s(i) = (CID(i) + 2) % 4 = 3, then r(i) = CID(i) % 4 = 1. The reward for leaving s3 is always equal to 1. Therefore, rewards are deterministic and do not depend on any particular action.

$$R = \begin{bmatrix} 2 \\ 3 \\ 0 \\ 1 \end{bmatrix}$$

3) The first state is s3. First-visit MC: Gt (of first s3) = r2 + r3 + r4 + r5 + r6 + r7 = 1 + 0 + 1 + 3 + 2 + 1 = 8 (since $\gamma = 1$). Every-visit MC = $\frac{1}{3}$(Gt of first s3 + Gt of second visit s3 + Gt of third visit) = $\frac{1}{3}$ [8 + (r4 + r5 + r6 + r7) + (r7)] = 1/3(8 + 7 + 1) = $\frac{16}{3}$ = 5.33.

Exercise 2

Question 0

The probability p = 0.8 + 0.02(9 - y) = 0.8 since y = 9, the discount factor γ = 0.8 + 0.02*y = 0.98, the reward state R_i, with i = z % 4 = 3 % 4 = 3, is R_3 (z = 3). The absorbing reward is 500 for R_3, else it is -50 for the rest of the absorbing states (R_0, R_1, R_2).

Question 1

1) I used the asynchronous value iteration method. I used value iteration because it works well with small datasets. It updates the value function and the policy in one step, which makes it less computationally expensive per iteration and simpler to implement. I also used asynchronous dp to store fewer versions of v, saving memory. I chose the threshold δ to be 0.001, which is good enough to stop updating the policy when the values are barely changing (by δ).

2) As evident in the images, following the policy at any grid would lead to the green absorbing state (R3 with the highest reward), and the closer a state is to it, the higher its value function.

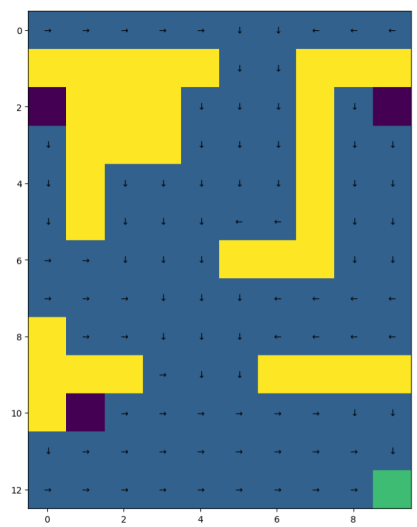


Figure 1: Optimal Policy for DP

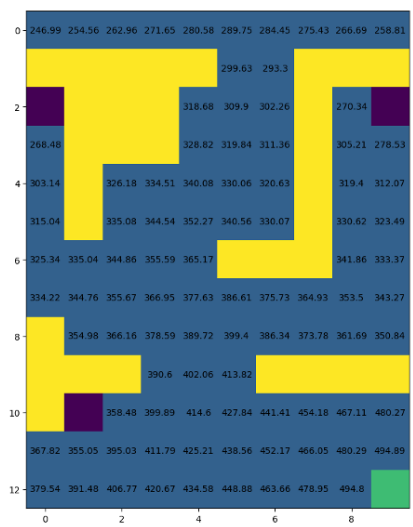


Figure 2: Optimal Value Function for DP

3) The higher the value of p is, the less stochastic and random the policy is. In this example, we have p=0.8, which is pretty high. This is why the optimal

policy in Fig. 1 is to take the action that would take the agent closer to R3; that action has a high probability of actually being executed.

If p=0.25, then taking any action could lead to moving N, E, W, or S with equal probability. If p<0.25 (for e.g., p=0.1), then the chosen action would have the lowest probability to happen. For e.g., if the agent chooses to go N, then it will have the lowest probability of going N. Both of these cases would lead to random and unexpected behavior and policies, as shown in Figures 3 and 5, respectively. Interestingly, in Figure 3, all policies are to move upwards. This is probably because all actions resulted in equal value functions, so the argmax(a) function returned a0 (going N). Additionally, the value functions of being in any state are pretty random in both cases and would change in different runs; however, they are higher for states closer to R3.

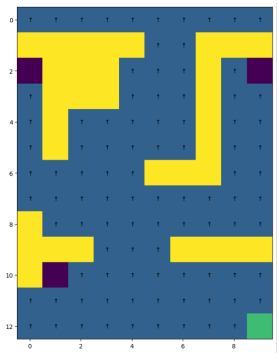


Figure 3: Policy when p=0.25

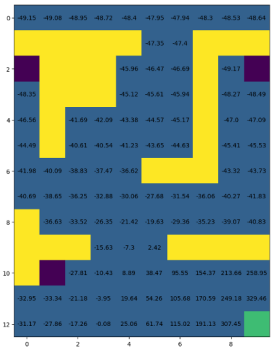


Figure 4: Value Function when p=0.25

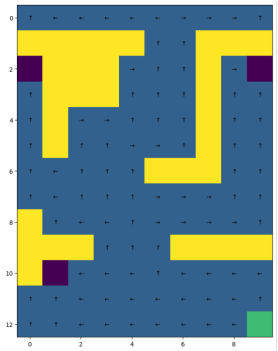


Figure 5: Policy when p=0.1

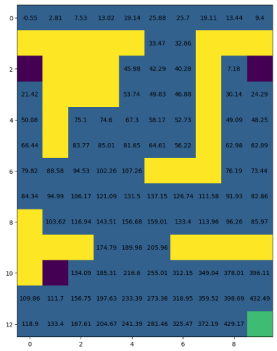


Figure 6: Value Function when p=0.1

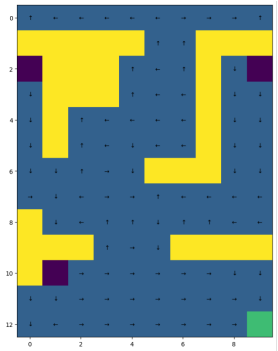


Figure 7: Policy when γ=0.1

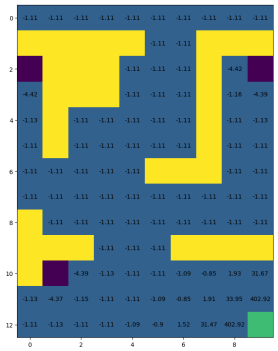


Figure 8: Value Function when γ=0.1

For γ, the higher it is, the more far-sighted the agent is. In our case, γ=0.98. This means that the value function depends on the long-term rewards, which is helpful in the case of mazes where the absorbing state farthest from the starting points gives the biggest reward. For smaller values of γ, the values

of states closest to absorbing states are more accurate, whereas other states would have the same value (around -1.11 for γ=0.1) since that is the expected reward (almost equal to the immediate reward) they would get if they chose any action. Therefore, the policy would be more random in states farthest from the absorbing states. Check figures 7 and 8 for γ=0.1.

Question 2

1) I used incremental MC control for this exercise. I chose every-visit MC with incremental means for policy evaluation because it gives more data points to estimate the value function which is practical when we have a finite number of value updates. Additionally, the incremental means method works without having to store the sample traces. I also used ϵ-greedy for policy improvement because it allows for exploration. I chose the exploration rate ϵ=0.1 and the learning rate α=0.01 because they showed the best results (check number 5 for ϵ choice). Higher values of alpha were overshooting, while lower values required more episodes to converge. I also chose the number of episodes n=10,000 since it showed better results compared to lower values (n=1000).

2) It is clear that there is a variability between different runs of MC, according to figures 9-12.

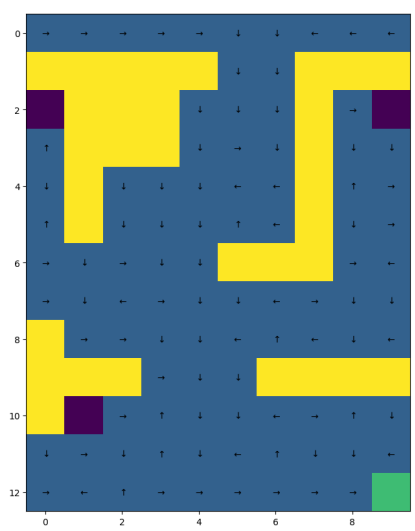


Figure 9: Optimal Policy for MC (run 1)

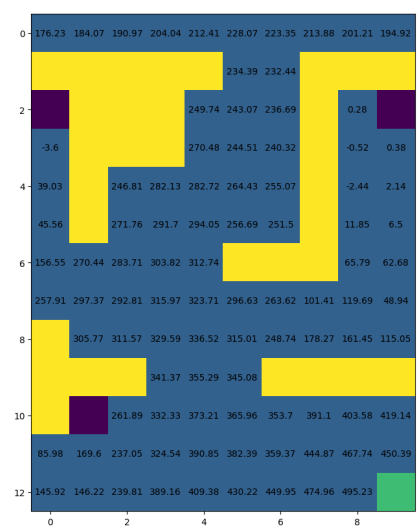


Figure 10: Optimal Value Function for MC (run 1)

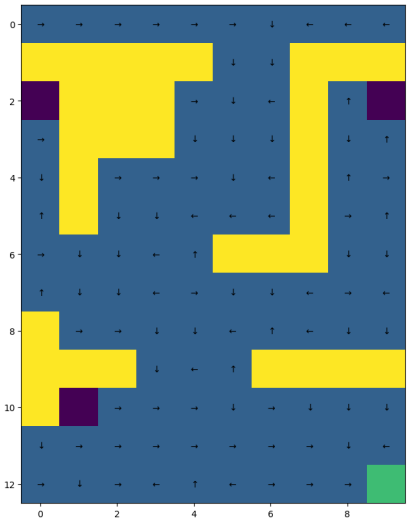


Figure 11: Optimal Policy for MC (run 2)

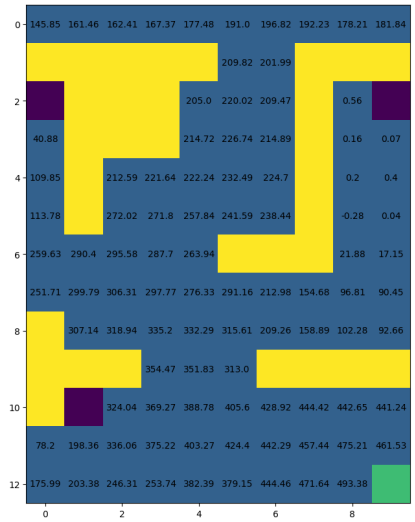


Figure 12: Optimal Value Function for MC (run 2)

3) The variability in learning is due to the total sum of discounted rewards, or returns, of the states, which vary between different episodes due to the stochastic nature of the model.

To choose a sufficient number of replications of the MC-learning runs, I first tried different possible values of *num_replications* (1, 2, 5, 10, 15, 20, 25). For every value, I plotted the mean of the total rewards across different replications $\pm$ std for every episode. Results are shown in the figures below (13-19). As evident, the range $[\text{mean}-\text{std}, \text{mean}+\text{std}]$ decreases with the increase of the number of replications, and this decrease is significant until *num_replications*=10; after that, the decrease becomes slight.

To make sure that these results did not happen by chance, I ran two different Multi MC-learning agents for each *num_replications*=2, 5, 10, 15, and 20, respectively. I didn't check for *num_replications*=25 since the learning curves between 20 and 25 look very similar, and doing two agents with 25 reps each is very expensive, computation- and time-wise. I also checked for *num_replications*=2 and 5, just for a visual comparison's sake. I then compared the absolute mean difference for each num_replications between the two agents. For example, for *num_replications*=10, I averaged the total rewards for the two different agents (each replicated 10 times) and calculated the absolute difference between the two mean rewards. This difference decreased from around 300 (for *num_replications*=2) to about 100 (for *num_replications*=5) and finally to less than 50 (for *num_replications*$\geq$10). The absolute differences in mean reward between 2 different Multi MC-learning agents for each *num_replications* are shown in fig. 20-24.

When the difference in the absolute mean total rewards between the two

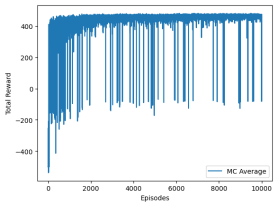


Figure 13: Learning Curve with *num_replications*=1

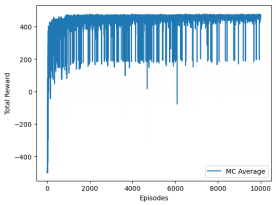


Figure 14: Learning Curve with *num_replications*=2

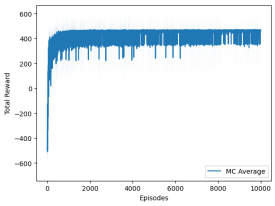


Figure 15: Learning Curve with *num_replications*=5

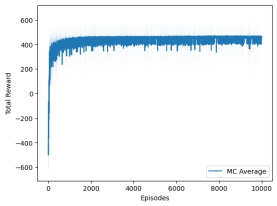


Figure 16: Learning Curve with *num_replications*=10

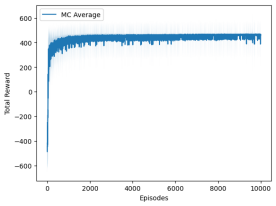


Figure 17: Learning Curve with *num_replications*=15

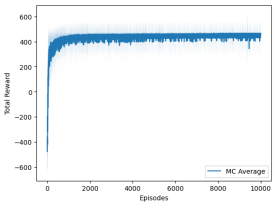


Figure 18: Learning Curve with *num_replications*=20

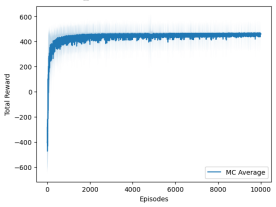


Figure 19: Learning Curve with *num_replications*=25

agents became small (≤ 50), I considered that the *num_replications* was sufficient. Therefore, I chose *num_replications*=10 because it balances between the sufficiently low variance and reduced computations and time, compared to higher values of *num_replications*. Fig. 25 shows the max difference between the two MC agents (for each *num_replications*) in the last 100 episodes (to reduce noise).

4) Fig. 26 shows the learning curve of a Multi MC agent with 10 replications/learning runs. It shows the average total rewards across the 10 runs (y-axis) over 10000 episodes (x-axis).

5) Higher ϵ values increase exploration, and lower values act greedily based on the state-action values. We need an ϵ value that balances between exploration and choosing a policy greedily. For ϵ=0.9, the policy was random due to very high exploration, and the learning curve shows low rewards (see Figures 27-29).

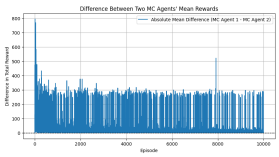


Figure 20: Absolute Mean Difference between Two Agents with *num_replications*=2

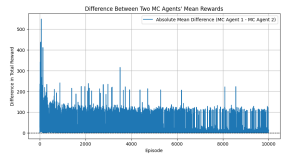


Figure 21: Absolute Mean Difference between Two Agents with *num_replications*=5

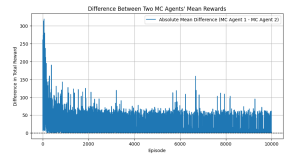


Figure 22: Absolute Mean Difference between Two Agents with *num_replications*=10

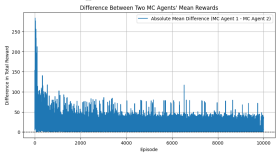


Figure 23: Absolute Mean Difference between Two Agents with *num_replications*=15

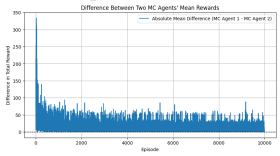


Figure 24: Absolute Mean Difference between Two Agents with *num_replications*=20

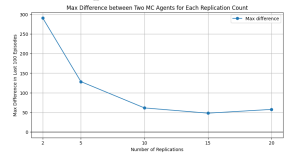


Figure 25: Max Absolute Mean Difference between Two MC Agents for Each *num_replications*

Similar to question 1 part 4, higher values of γ (far-sighted) yield more accurate state values in our case since the **major** rewards are sparse and are only obtained at the absorbing states. Figures 30-32 show the effect of small γ (=0.1).

Fig. 33 shows the combinations of ϵ and γ that maximize the performance of MC (over 10 replications).

Question 3

1) I used the forward view SARSA(0) algorithm with ϵ-greedy because it is simple and clean, especially for a small to medium number of states, such as in this problem. Like MC, I used ϵ=0.1 and number of episodes n=10000 for similar reasons. Unlike MC, I chose α=0.25 since it showed better results compared to α=0.01.

2) Check Fig. 34-35

3) Check Fig. 36

4) Varying the exploration rate ϵ in TD learning has similar effects as varying it in MC; i.e., values that are too high will result in a more random policy and lower returns (check fig. 37-39 for results of ϵ=0.9). Values that are too low (e.g. ϵ=0, check fig. 40-42) would have higher reward, but the policy outside of the main path (from starting points to R3) is less optimized due to low exploration.

Varying the learning rate α would result in a lot of spikes and noises in the learning curve if α is too high and in slower convergence if it is too low. Check figures 43-51 for the effects of α=0.9, 0.5, and 0.01 respectively.

Question 4

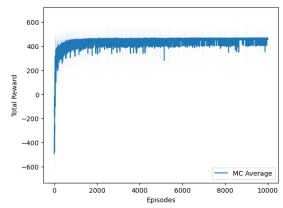


Figure 26: Learning Curve for *num_replications*=10

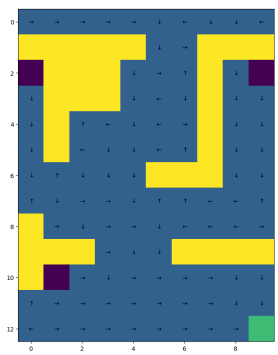


Figure 27: Optimal Policy for ϵ=0.9 (γ=0.98)

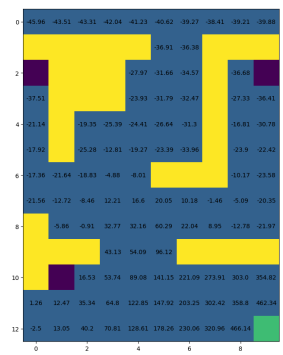


Figure 28: Optimal Value Function for ϵ=0.9 (γ=0.98)

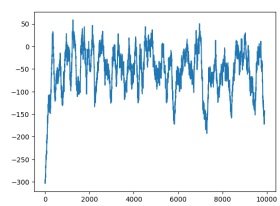


Figure 29: Learning Curve for ϵ=0.9 (γ=0.98)

1) Check Fig. 52.

2) According to the graph in Fig. 52, TD converges faster than MC, the variability across the 10 different replications is much less than that of MC (compare red area around the curve with blue area), and at around n=10000, both TD and MC converge to approximately the same value of MSE. This is consistent with the theory of the two approaches. Since TD updates its value function at every step, unlike MC, which updates it after the episode terminates, it converges faster. Also, since the update of MC depends on the total discounted sum of return, it has a higher variability, which is demonstrated in the graph as a wide blue area around its curve. Finally, according to theoretical proofs, both methods are guaranteed to converge as n goes to infinity.

3) Check figures 53-55.

4) This plot shows the correlation between value function estimation (implied from the MSE) and rewards. For both MC and TD learners, high rewards were obtained (more than 400), both when the MSE error was high and low. It is also observed that the reward could be low even if the MSE is low. This implies that a good value function estimation is not necessary for obtaining a

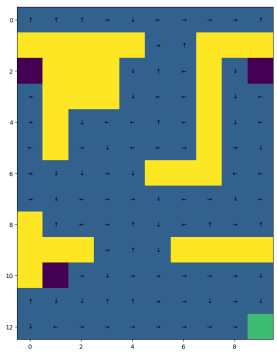


Figure 30: Optimal Policy for γ=0.1 (ϵ=0.1)

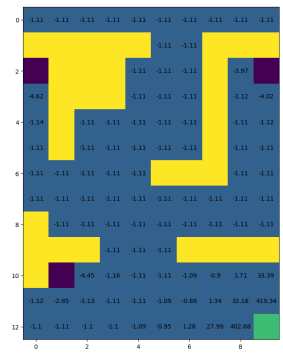


Figure 31: Optimal Value Function for γ=0.1 (ϵ=0.1)

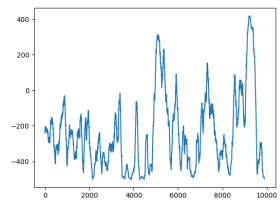


Figure 32: Learning Curve for γ=0.1 (ϵ=0.1)

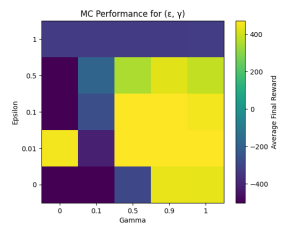


Figure 33: Performance of MC across different combinations of ϵ and γ

high reward. This is because MSE measures errors across all states, whereas the total reward is dependent on the states that are taken in the trajectory. Hence, the agents could better estimate more frequently visited states.

Since MC results in higher variance compared to TD, there should be more spread along the vertical axis (MSE), as evident in Fig. 55, unlike TD.

Higher exploration rates ϵ would lead to less reward but better estimates due to exploration, while high learning rates α would increase variance in value estimates, widening the scatter.

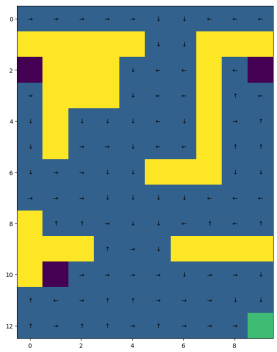


Figure 34: Optimal Policy for SARSA(0)

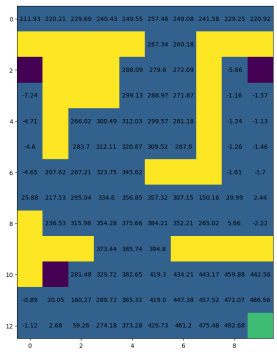


Figure 35: Optimal Value Function for SARSA(0)

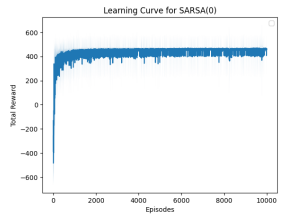


Figure 36: Learning Curve for SARSA(0) across 10 replications

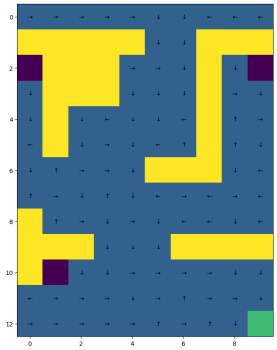


Figure 37: Optimal Policy for SARSA(0) with ϵ=0.9 (α=0.01)

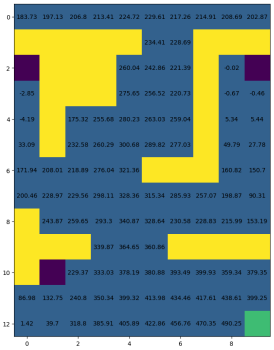


Figure 38: Optimal Value Function for SARSA(0) with ϵ=0.9 (α=0.01)

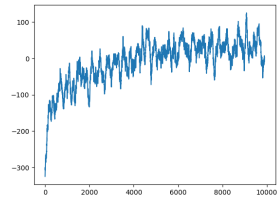


Figure 39: Learning Curve for SARSA(0) with ϵ=0.9 (α=0.01)

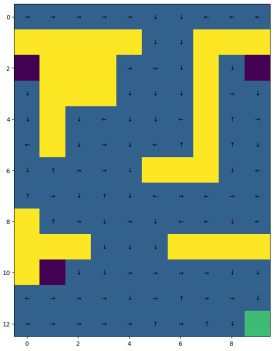


Figure 40: Optimal Policy for SARSA(0) with ϵ=0 (α=0.1)

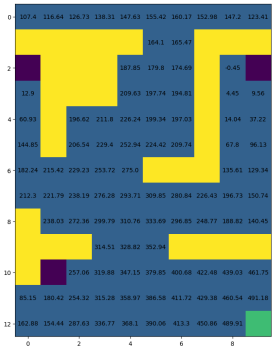


Figure 41: Optimal Value Function for SARSA(0) with ϵ=0 (α=0.1)

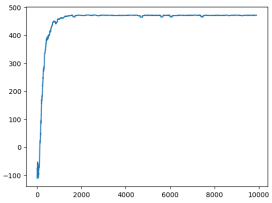


Figure 42: Learning Curve for SARSA(0) with ϵ=0 (α=0.1)

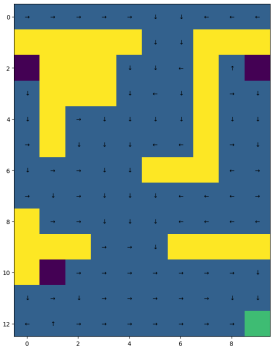


Figure 43: Optimal Policy for SARSA(0) with α=0.9 (ϵ=0.1)

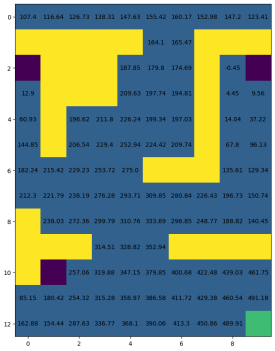


Figure 44: Optimal Value Function for SARSA(0) with α=0.9 (ϵ=0.1)

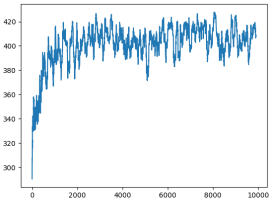


Figure 45: Learning Curve for SARSA(0) with α=0.9 (ϵ=0.1)

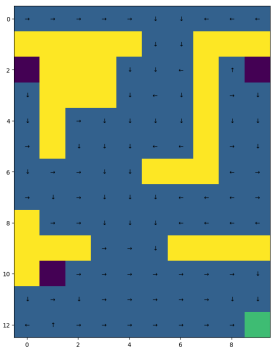


Figure 46: Optimal Policy for SARSA(0) with α=0.5 (ϵ=0.1)

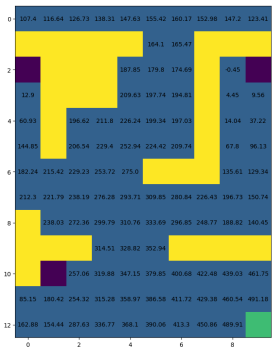


Figure 47: Optimal Value Function for SARSA(0) with α=0.5 (ϵ=0.1)

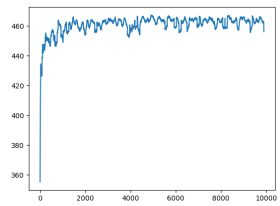


Figure 48: Learning Curve for SARSA(0) with α=0.5 (ϵ=0.1)

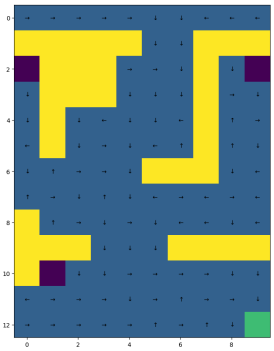


Figure 49: Optimal Policy for SARSA(0) with α=0.01 (ϵ=0.1)

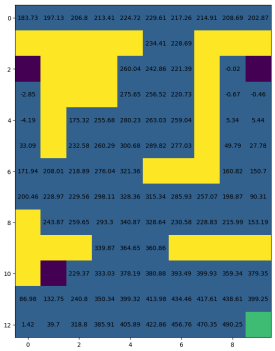


Figure 50: Optimal Value Function for SARSA(0) with α=0.01 (ϵ=0.1)

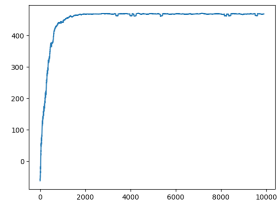


Figure 51: Learning Curve for SARSA(0) with α=0.01 (ϵ=0.1)

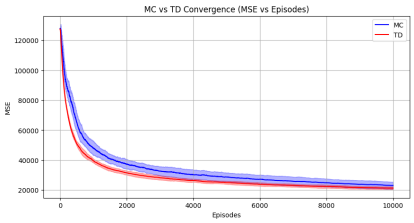


Figure 52: MC vs TD Convergence (MSE across Episodes)

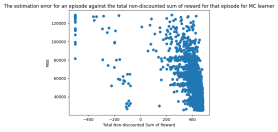


Figure 53: Scatter Plot for MC Learner

Figure 54: Scatter Plot for TD Learner

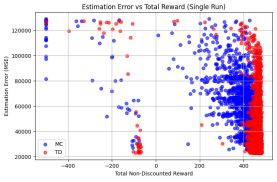


Figure 55: Scatter Plot for both MC and TD learners